

UNIVERSIDAD DE SANTIAGO DE CHILE
FACULTAD DE INGENIERÍA
Departamento de Ingeniería Informática



Tarea 1: Metaheurísticas

Diseño de Componentes para el Problema del Clique Máximo

Integrantes: Ricardo Riveros

Gonzalo Ahumada

Daniel Muñoz

Asignatura: Optimización en Ingeniería

Profesor: Mario Inostroza

Ayudante: Sebastián Aliaga

12 de junio de 2026

1. Descripción del Problema

Este trabajo da continuidad al estudio del Problema del Clique Máximo (*Maximum Clique Problem*, MCP) iniciado en el Informe 1, donde se abordó su resolución mediante un método clásico exacto (Programación Lineal Entera resuelta por Ramificación y Acotamiento). En esta segunda etapa se mantiene exactamente la misma definición del problema, pero se cambia el paradigma de resolución hacia metaheurísticas de búsqueda local.

Formalmente, dado un grafo no dirigido $G = (V, E)$ con $|V| = n$ vértices, un *clique* es un subconjunto $C \subseteq V$ tal que todo par de vértices distintos $u, v \in C$ satisface $\{u, v\} \in E$; es decir, C induce un subgrafo completo de G . El MCP consiste en encontrar un clique C^* de cardinalidad máxima $\omega(G) = |C^*|$, denominado *número de clique* de G (Pardalos & Xue, 1994). El MCP pertenece a la clase de problemas NP-duros, y se ha demostrado que ni siquiera admite un algoritmo de aproximación con factor constante en tiempo polinomial, salvo que $P = NP$ (Bomze et al., 1999; Pardalos & Xue, 1994).

En el Informe 1, la formulación ILP del MCP (variables binarias $x_i \in \{0, 1\}$ indicando si el vértice i pertenece al clique, restricciones $x_i + x_j \leq 1$ para cada par no adyacente) fue resuelta de forma exacta mediante Branch & Bound (HiGHS) sobre la instancia *keller4* (171 vértices, 9 435 aristas), obteniendo $\omega = 11$ en 12–18 segundos. Sin embargo, ese mismo experimento mostró la fragilidad del enfoque exacto: el solver GLPK, sobre el modelo idéntico, no logró probar optimalidad en 300 segundos. Para instancias más densas o de mayor tamaño, el árbol de Branch & Bound crece de forma exponencial y la resolución exacta se vuelve computacionalmente inviable en tiempos razonables.

Por ello, esta tarea inicia el diseño de un enfoque metaheurístico de búsqueda local para el MCP, que renuncia a la garantía de optimalidad a cambio de escalabilidad a instancias grandes. Concretamente, se especifican los componentes fundamentales —representación de soluciones, función objetivo, manejo de restricciones y operador de generación de vecindario— que servirán de base para metaheurísticas de trayectoria

como Búsqueda Tabú, Recocido Simulado, GRASP+VNS o Búsqueda Local Iterada (Battiti & Protasi, 2001; Katayama et al., 2005; Pullan & Hoos, 2006).

2. Representación de la Solución

Una solución se representa directamente como un subconjunto de vértices

$$S \subseteq V, \quad S = \{v_1, v_2, \dots, v_k\},$$

que constituye un clique de G , codificado como una lista (o conjunto) de índices de vértices, sin orden ni repetición, de longitud variable $k = |S| \leq n$.

A diferencia de la representación binaria $x \in \{0, 1\}^n$ utilizada en la formulación ILP del Informe 1 (donde $x_i = 1 \Leftrightarrow i \in S$), aquí se opta por una representación *explícita de longitud variable*: S codifica directamente, sin necesidad de un decodificador, el conjunto de vértices que conforman el clique. Esta elección, habitual en metaheurísticas de búsqueda local para el MCP (Battiti & Protasi, 2001; Katayama et al., 2005; Pullan & Hoos, 2006), tiene dos ventajas centrales:

- El tamaño de la codificación es proporcional al tamaño del clique candidato ($k \ll n$ típicamente), evitando recorrer un vector de longitud n en cada operación.
- Permite mantener, de forma incremental, el *conjunto de candidatos* de S , esencial para el operador de vecindad del punto 6.

Sea $N(v) = \{u \in V : \{u, v\} \in E\}$ el vecindario abierto del vértice v en G . Intuitivamente, interesa identificar qué vértices fuera de S todavía “caben” en el clique sin romperlo, es decir, sin introducir un par no adyacente. Formalmente, se define el **conjunto de candidatos** de una solución S como

$$C(S) = \left(\bigcap_{u \in S} N(u) \right) \setminus S, \quad (1)$$

es decir, el conjunto de vértices fuera de S que son adyacentes a *todos* los vértices de S . Cualquier $v \in C(S)$ puede incorporarse a S sin romper la propiedad de clique. Para $S = \emptyset$ se define $C(\emptyset) = V$.

3. Función Objetivo

Dado que la representación garantiza que toda solución S del espacio de búsqueda es, por construcción, un clique válido de G , la función objetivo es simplemente la cardinalidad del conjunto:

$$f(S) = |S|, \quad \text{maximizar } f(S). \quad (2)$$

Esta expresión es equivalente a la función objetivo del modelo ILP del Informe 1 ($\text{máx} \sum_{i \in V} x_i$), pero no requiere sumar sobre un vector indicador de tamaño n : $f(S)$ se obtiene en tiempo $O(1)$ manteniendo un contador junto con S . Como toda solución visitada es factible (Sección 4), $f(S)$ está siempre bien definida y no es necesario introducir términos de penalización ni una función objetivo aumentada.

4. Manejo de Restricciones

La única restricción estructural del MCP es la *restricción de adyacencia*: para todo par $u, v \in S$ con $u \neq v$, debe cumplirse $\{u, v\} \in E$ (equivalentemente, ningún par de vértices no adyacentes de $\bar{E}$ puede pertenecer simultáneamente a S , análogo a la restricción $x_i + x_j \leq 1$ de la formulación ILP del Informe 1).

En este diseño, dicha restricción se maneja *por construcción* y no mediante penalización: en lugar de admitir cualquier subconjunto de V y penalizar las aristas faltantes, el espacio de búsqueda se restringe *desde el inicio* a los subgrafos completos de G , es decir, a la familia de todos los cliques de G ,

$$\Omega = \{S \subseteq V : S \text{ es un clique de } G\},$$

y los tres operadores de movimiento definidos en la Sección 6 (ADD, DROP, SWAP) se diseñan de modo que $S \in \Omega \Rightarrow S' \in \Omega$ para todo vecino $S' \in N(S)$. Esto tiene las siguientes consecuencias:

- Toda solución visitada durante la búsqueda es factible; no existe región infactible que explorar ni reparar.

- No se requieren penalizaciones, multiplicadores de Lagrange ni heurísticas de reparación, a diferencia de enfoques con representación binaria y función objetivo penalizada (frecuentes en algoritmos genéticos para el MCP).
- El precio de esta factibilidad estricta es que la vecindad queda “cerrada” dentro de Ω . No obstante, la conectividad del espacio de búsqueda está garantizada: a partir de cualquier $S \in \Omega$, una secuencia de movimientos DROP permite llegar al clique vacío $S = \emptyset$, donde $C(\emptyset) = V$ permite agregar cualquier vértice y así alcanzar, en principio, cualquier otro clique de G .

5. Ejemplo de Instancia, Representación y Función Objetivo

Para ilustrar los conceptos anteriores se utiliza la instancia ilustrativa $G = (V, E)$ de la Figura 1, con $V = \{1, \dots, 7\}$ y

$$E = \{\{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{2, 4\}, \{3, 4\}, \\ \{3, 5\}, \{4, 5\}, \{5, 6\}, \{5, 7\}, \{6, 7\}, \{2, 6\}\}, \quad |E| = 12.$$

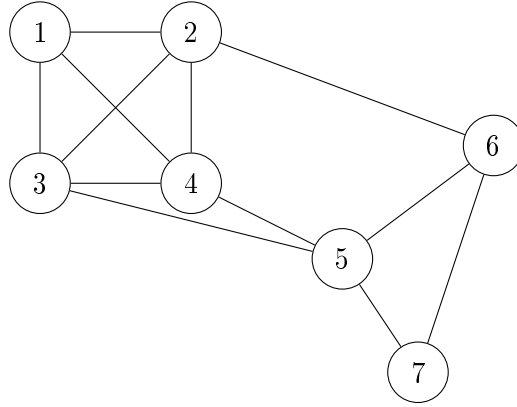


Figura 1: Grafo de ejemplo $G = (V, E)$, $|V| = 7$, $|E| = 12$.

Las listas de adyacencia son:

$$\begin{aligned} N(1) &= \{2, 3, 4\}, & N(2) &= \{1, 3, 4, 6\}, & N(3) &= \{1, 2, 4, 5\}, & N(4) &= \{1, 2, 3, 5\}, \\ N(5) &= \{3, 4, 6, 7\}, & N(6) &= \{2, 5, 7\}, & N(7) &= \{5, 6\}. \end{aligned}$$

Ejemplo 1: solución no maximal. Sea $S = \{1, 2, 3\}$. Se verifica que $\{1, 2\}, \{1, 3\}, \{2, 3\} \in E$, por lo que S es un clique válido y

$$f(S) = |S| = 3.$$

El conjunto de candidatos es $C(S) = (N(1) \cap N(2) \cap N(3)) \setminus S = \{4\}$, ya que 4 es adyacente a 1, 2 y 3. Esto indica que S puede mejorarse agregando el vértice 4.

Ejemplo 2: solución óptima de la instancia ilustrativa. Sea $S' = \{1, 2, 3, 4\}$ (resultado de aplicar $\text{ADD}(S, 4)$). Todas las $\binom{4}{2} = 6$ aristas requeridas están presentes, por lo que S' es un clique y

$$f(S') = |S'| = 4.$$

Su conjunto de candidatos es $C(S') = (N(1) \cap N(2) \cap N(3) \cap N(4)) \setminus S' = \emptyset$: ningún vértice restante es adyacente a los cuatro vértices de S' , por lo que S' es *maximal* respecto a ADD . Puede verificarse que $\omega(G) = 4$, es decir, S' es además óptima.

Ejemplo 3: solución maximal subóptima. Sea $S_4 = \{3, 4, 5\}$. Es un clique válido ($\{3, 4\}, \{3, 5\}, \{4, 5\} \in E$) con $f(S_4) = 3$, pero $C(S_4) = (N(3) \cap N(4) \cap N(5)) \setminus S_4 = \emptyset$: ningún vértice puede agregarse directamente. Sin embargo S_4 *no* es óptima ($f(S_4) = 3 < \omega(G) = 4$). Este caso —un óptimo local bajo el operador ADD — motiva la inclusión del operador SWAP en la vecindad, desarrollado en la Sección 6.

6. Algoritmo de Generación de Soluciones Vecinas

6.1. Definición de la vecindad

La vecindad $N(S)$ de una solución $S \in \Omega$ se define como la unión de tres tipos de movimientos, todos cerrados en Ω (Sección 4): agregar un vértice al clique, quitar un vértice del clique, o intercambiar un vértice de S por otro fuera de S . Formalmente:

- **ADD:** para cada $v \in C(S)$, $S' = S \cup \{v\}$ (incrementa f en 1).

- **DROP**: para cada $u \in S$, $S' = S \setminus \{u\}$ (decrementa f en 1; todo subconjunto de un clique es un clique).
- **SWAP**: para cada $v \notin S$ con $\deg_S(v) = |N(v) \cap S| = |S| - 1$, sea $\{u\} = S \setminus N(v)$ el único vértice de S no adyacente a v ; entonces $S' = (S \setminus \{u\}) \cup \{v\}$ (mantiene f constante, pero cambia $C(S)$, permitiendo escapar de óptimos locales como el del Ejemplo 3).

6.2. Pseudocódigo

El Algoritmo 1 recorre una sola vez $V \setminus S$ utilizando una matriz de adyacencia A (con $A[v][u] = 1 \Leftrightarrow \{v, u\} \in E$), clasificando cada vértice externo según su grado dentro de S , $\deg_S(v) = |N(v) \cap S|$.

Idea clave. La regla de clasificación es simple: si v es adyacente a *todo* S ($\deg_S(v) = |S|$), es candidato ADD; si a v le falta exactamente un vértice de S para ser adyacente a todos ($\deg_S(v) = |S| - 1$), entonces v habilita un movimiento SWAP con ese vértice faltante. En cualquier otro caso, v no contribuye a $N(S)$.

Algorithm 1 Generación de la vecindad $N(S)$ para el MCP

Require: Grafo $G = (V, E)$ (matriz de adyacencia A), clique factible $S \subseteq V$

Ensure: $N(S) = N_{\text{add}}(S) \cup N_{\text{drop}}(S) \cup N_{\text{swap}}(S)$

```

1:  $C(S) \leftarrow \emptyset$ ;    $N_{\text{swap}}(S) \leftarrow \emptyset$ 
2: for cada  $v \in V \setminus S$  do
3:    $d \leftarrow |N(v) \cap S|$   $\triangleright \deg_S(v)$ , calculado en  $O(|S|)$  con  $A$ 
4:   if  $d = |S|$  then
5:      $C(S) \leftarrow C(S) \cup \{v\}$   $\triangleright v$  adyacente a todo  $S \Rightarrow$  candidato ADD
6:   else if  $d = |S| - 1$  then
7:      $\{u\} \leftarrow S \setminus N(v)$   $\triangleright$  único  $u \in S$  no adyacente a  $v$  (existe pues  $d = |S| - 1$ )
8:      $N_{\text{swap}}(S) \leftarrow N_{\text{swap}}(S) \cup \{(S \setminus \{u\}) \cup \{v\}\}$   $\triangleright$  intercambia  $u$  por  $v$ 
9:   end if
10: end for
11:  $N_{\text{add}}(S) \leftarrow \{S \cup \{v\} : v \in C(S)\}$ 
12:  $N_{\text{drop}}(S) \leftarrow \{S \setminus \{u\} : u \in S\}$ 
13: return  $N_{\text{add}}(S) \cup N_{\text{drop}}(S) \cup N_{\text{swap}}(S)$ 

```

6.3. Complejidad algorítmica

Sea $n = |V|$ y $k = |S|$. Con representación de matriz de adyacencia (consulta $O(1)$), el cálculo de $d = |N(v) \cap S|$ y, en su caso, del vértice $u = S \setminus N(v)$ toma $O(k)$ por cada $v \in V \setminus S$. El bucle principal (líneas 2–9) cuesta entonces

$$O((n - k) \cdot k) = O(nk).$$

La construcción de $N_{\text{add}}(S)$ es $O(|C(S)|) \subseteq O(n)$, y la de $N_{\text{drop}}(S)$ es $O(k)$. Por lo tanto, la complejidad temporal total del Algoritmo 1 es

$$O(nk) = O(n^2) \text{ en el peor caso (cuando } k = \Theta(n)),$$

y $O(n)$ cuando k es constante u $O(1)$, situación habitual en cliques pequeños frente a n .

Complejidad espacial. Conviene distinguir dos formas de devolver $N(S)$:

- *Vecinos materializados:* si cada $S' \in N(S)$ se almacena como un conjunto completo (copia de S con un vértice agregado, eliminado o intercambiado), cada copia ocupa $O(k)$ y existen $O(n)$ vecinos, por lo que el espacio total es $O(nk)$.
- *Movimientos compactos:* en lugar de copiar S , cada vecino se describe mediante un movimiento $\text{ADD}(v)$, $\text{DROP}(u)$ o $\text{SWAP}(u, v)$, es decir, uno o dos índices de vértice ($O(1)$ por movimiento). Como hay $O(n)$ movimientos, el espacio adicional es $O(n)$. Aplicar un movimiento sobre S (para obtener S' y actualizar f , $C(S')$ y los grados $\deg_{S'}(\cdot)$) toma $O(k)$.

En la implementación se recomienda la representación por movimientos compactos, que mantiene el espacio adicional en $O(n)$.

6.4. Ejemplo de aplicación de la función de vecindad

Se aplica el Algoritmo 1 a $S_4 = \{3, 4, 5\}$ (Ejemplo 3, $f(S_4) = 3$, $C(S_4) = \emptyset$) sobre el grafo de la Figura 1. Para cada $v \in V \setminus S_4 = \{1, 2, 6, 7\}$ se calcula $\deg_{S_4}(v)$:

$$\deg_{S_4}(1) = |N(1) \cap S_4| = |\{3, 4\}| = 2 = k - 1 \Rightarrow \text{SWAP: } u = 5, S' = \{1, 3, 4\}$$

$$\deg_{S_4}(2) = |N(2) \cap S_4| = |\{3, 4\}| = 2 = k - 1 \Rightarrow \text{SWAP: } u = 5, S' = \{2, 3, 4\}$$

$$\deg_{S_4}(6) = |N(6) \cap S_4| = |\{5\}| = 1 \Rightarrow (\text{no clasifica})$$

$$\deg_{S_4}(7) = |N(7) \cap S_4| = |\{5\}| = 1 \Rightarrow (\text{no clasifica})$$

con $k = |S_4| = 3$. No hay candidatos ADD ($C(S_4) = \emptyset$), por lo que $N_{\text{add}}(S_4) = \emptyset$. Los movimientos DROP eliminan cada vértice de S_4 :

$$N_{\text{drop}}(S_4) = \{\{4, 5\}, \{3, 5\}, \{3, 4\}\}.$$

Los movimientos SWAP encontrados son $N_{\text{swap}}(S_4) = \{\{1, 3, 4\}, \{2, 3, 4\}\}$. En total,

$$N(S_4) = \{\{4, 5\}, \{3, 5\}, \{3, 4\}, \{1, 3, 4\}, \{2, 3, 4\}\}, \quad |N(S_4)| = 5.$$

Aunque ningún vecino mejora $f(S_4) = 3$ de inmediato, el vecino $\{2, 3, 4\}$ —obtenido por SWAP— tiene $C(\{2, 3, 4\}) = \{1\} \neq \emptyset$ (vértice 1 es adyacente a 2, 3 y 4). Una metaheurística que acepte este movimiento lateral podrá, en la siguiente iteración, aplicar ADD(1) y alcanzar $S^* = \{1, 2, 3, 4\}$ con $f(S^*) = 4 = \omega(G)$, el óptimo de la instancia ilustrativa. Este ejemplo ilustra cómo el operador SWAP permite escapar de óptimos locales del operador ADD sin abandonar el conjunto factible Ω .

6.5. Rol de $N(S)$ dentro de una metaheurística

El Algoritmo 1 define únicamente la *estructura de vecindad* $N(S)$: qué movimientos son alcanzables desde una solución S . Recorrer $N(S)$ aceptando siempre el mejor vecino corresponde a una búsqueda local de descenso/ascenso, que queda atrapada en óptimos locales como S_4 del Ejemplo 3.

Una metaheurística construida sobre esta misma vecindad agrega, además, una **regla de selección y aceptación de movimientos**, cuya forma depende del método:

- **Búsqueda Tabú**: se permite moverse a vecinos que empeoran f (p. ej., DROP o SWAP hacia S_4), pero se prohíbe temporalmente revertir el último movimiento (lista tabú sobre los vértices recién agregados/eliminados), salvo que se cumpla un criterio de aspiración.
- **Recocido Simulado**: un vecino $S' \in N(S)$ que empeora f se acepta con probabilidad $e^{(f(S')-f(S))/T}$, con T decreciente según un esquema de enfriamiento, permitiendo escapar de óptimos locales sin prohibir movimientos explícitamente.
- **VNS (Variable Neighborhood Search)**: N_{add} , N_{drop} y N_{swap} pueden tratarse como estructuras de vecindad distintas N_1, N_2, N_3 ; si la búsqueda en N_1 no mejora S , se conmuta a N_2 , y así sucesivamente, antes de reiniciar desde N_1 .

En todos los casos, el costo por iteración está dominado por la generación de $N(S)$, por lo que su complejidad $O(nk)$ constituye la unidad de costo básica de cualquier metaheurística construida sobre esta vecindad.

Referencias

- Battiti, R., & Protasi, M. (2001). Reactive local search for the maximum clique problem. *Algorithmica*, 29(4), 610-637. <https://doi.org/10.1007/s004530010074>
- Bomze, I. M., Budinich, M., Pardalos, P. M., & Pelillo, M. (1999). The maximum clique problem. *Handbook of Combinatorial Optimization*, 4, 1-74.
- Katayama, K., Hamamoto, A., & Narisawa, H. (2005). An effective local search for the maximum clique problem. *Information Processing Letters*, 95(5), 503-511. <https://doi.org/10.1016/j.ipl.2005.05.010>
- Pardalos, P. M., & Xue, J. (1994). The maximum clique problem. *Journal of Global Optimization*, 4(3), 301-328. <https://doi.org/10.1007/bf01098364>
- Pullan, W., & Hoos, H. H. (2006). Dynamic local search for the maximum clique problem. *Journal of Artificial Intelligence Research*, 25, 159-185. <https://doi.org/10.1613/jair.1815>